Day 14 Learning (31/03/2026)

#Spring Boot Project Setup and Structure:

- Introduction to creating a Spring Boot project using Spring Initializr.
- Explanation of Maven and its role in managing project dependencies.
- Understanding the project structure and key files like pom.xml.

Inversion of Control (IoC) and IoC Container:

- Traditional Java requires manual instantiation of objects (e.g., Car c = new Car()).
- Spring Boot uses Inversion of Control (IoC) where the creation of objects is delegated to the IoC container.
- IoC container (also called ApplicationContext) holds and manages all the instances (beans) in the project.
- Objects are obtained from the container on demand, enabling loose coupling and easier management.

Component Scanning and @Component Annotation

- IoC container scans specified base packages to identify which classes should be managed.
- Only classes annotated with @Component (or its specialized variants) are registered as beans within the IoC container.
- This scanning is limited to the base package and its sub-packages; classes outside these are not scanned or registered.

Beans and Bean Creation

- In Spring terminology, an object managed by the IoC container is called a bean.
- Annotating a class with @Component marks it as a bean candidate.
- Beans can be injected wherever needed without manual instantiation.

Dependency Injection and @Autowired

- Instead of creating dependencies manually, Spring Boot supports dependency injection.
- The @Autowired annotation on fields instructs Spring to inject the required bean automatically.
- Example: A Car class can have a Dog class dependency injected without new Dog().

- If @Autowired is missing and manual instantiation is also omitted, NullPointerException occurs.

Spring Boot Application Class and @SpringBootApplication Annotation

- The main entry point of a Spring Boot application is the class annotated with @SpringBootApplication.
- This annotation is a composite of three key annotations:

Annotation	**Purpose**
@ComponentScan	Enables scanning of components (i.e., @Component) in the base package and sub-packages.
@EnableAutoConfiguration	Enables automatic configuration based on dependencies present in the classpath.
@Configuration	Indicates that the class can be used by Spring IoC container as a source of bean definitions.

- The base package is the package of the main application class, and only classes within this package and its sub-packages are scanned.

Automatic Configuration (@EnableAutoConfiguration)

- Automatically configures application components such as database connections based on dependencies (e.g., MongoDB) and properties files.
- Eliminates manual and repetitive configuration, accelerating development.

Specialized Stereotype Annotations

- Besides @Component, other annotations like @RestController exist.
- @RestController is a specialized version of @Component that additionally configures the class to handle REST API requests.

#Core Definitions and Comparisons:

Term/Annotation	Description
IoC Container / ApplicationContext	A container that manages the lifecycle and dependencies of beans (objects) in a Spring Boot application.
Bean	An object that is instantiated, assembled, and managed by the Spring IoC container.
@Component	Annotation indicating that a class is a Spring-managed bean eligible for component scanning.
@Autowired	Annotation used to inject dependencies automatically by Spring's IoC container.
@SpringBootApplication	Composite annotation combining @ComponentScan, @EnableAutoConfiguration, and @Configuration.
@EnableAutoConfiguration	Enables Spring Boot to automatically configure beans based on classpath dependencies and properties.
@ComponentScan	Instructs Spring where to scan for components (beans); limited to base package and sub-packages.

@RestController	Specialized @Component for RESTful web services, combining @Controller and @ResponseBody.

#Key Insights and Conclusions:

- **Inversion of Control (IoC) is fundamental** to Spring Boot's design, shifting object creation responsibility from the developer to the framework.
- The IoC container holds all beans, enabling easy dependency injection and management.
- Component scanning is package-sensitive: Only classes in the base package (and sub-packages) where the main application resides are scanned and registered as beans.
- **@SpringBootApplication simplifies configuration** by combining multiple essential annotations, facilitating component scanning and auto-configuration.
- **@Autowired allows automatic dependency injection**, reducing boilerplate code and preventing manual instantiation errors.
- Auto-configuration greatly reduces manual setup work, especially for integrating databases and other services.
- The framework promotes single-instance usage of beans, improving resource management and consistency across the application.

#Maven in Springboot:

Maven Definition:

- Maven is described as a **build automation tool** that simplifies the process of building projects and managing dependencies.

Build Automation:

- Maven automates tasks such as compiling source code, packaging it into JAR or WAR files, running tests, and deploying artifacts.

Dependency Management:

- Instead of manually downloading JAR files for external libraries (e.g., OpenCSV), Maven allows developers to declare dependencies in a **pom.xml** file.
- Maven then automatically downloads these libraries from its **central repository**.

Coordinates of Dependencies

- Each dependency is identified by three key coordinates:
    1. **Group ID** (company or organization name)
    2. **Artifact ID** (library or module name)
    3. **Version** (specific release/version of the library)

Using Maven Dependencies

- Developers search Maven repositories for required libraries, copy the dependency snippet, and paste it inside the project's **pom.xml**.
- Maven downloads and caches these JARs locally in the **.m2** directory, enabling reuse across multiple projects without re-downloading.

Maven Build Lifecycle Phases

The official Maven lifecycle includes multiple phases, which are:

- **validate** – checks the project is correct and all necessary information is available
- **compile** – compiles the source code
- **test** – runs unit tests
- **package** – packages the compiled code into JAR or WAR files
- **verify** – runs any additional checks on the results of integration tests
- **install** – installs the package into the local repository (.m2 folder)
- **deploy** – copies the final package to a remote repository for sharing

Running Maven Commands

- If Maven is installed on the system, commands like **mvn validate**, **mvn compile**, **mvn test**, and **mvn package** can be run directly.
- If Maven is not installed, the **Maven Wrapper (mvnw)** included in Spring Boot projects can be used to run Maven commands without a system-wide installation.
- **mvn clean** deletes the **target** directory to remove previous build artifacts.

Output and Usage of Built Artifacts

- The final JAR file is generated inside the **target** directory.
- This JAR can be shared and run on any machine with Java installed—no need for external servers like Tomcat because Spring Boot embeds the server inside the JAR.

Maven Build Lifecycle Phases (Definitions):

Phase	Description
validate	Checks correctness of project configuration and necessary info
compile	Compiles the source code
test	Runs unit tests
package	Packages compiled code into deployable format (JAR/WAR)
verify	Performs integration and additional tests
install	Installs the package into local Maven repository (.m2 folder)
deploy	Copies the package to a remote repository for sharing

Important Terms:

- **pom.xml:** XML file describing the project's dependencies, build configuration, and plugins.
- **Dependency Coordinates:** Group ID, Artifact ID, and Version uniquely identify a dependency.
- **Maven Local Repository (.m2):** Directory on the local machine where Maven stores downloaded JAR files.
- **Maven Wrapper (mvnw):** A script that allows running Maven commands without installing Maven globally.
- **Spring Boot JAR:** Self-contained executable JAR with embedded server, making deployment simple.

Key Takeaways:

- **Maven is essential** for managing builds and dependencies in Java projects, especially Spring Boot.
- Dependencies declared in **pom.xml** are automatically downloaded and cached, avoiding manual JAR management.
- **Maven's lifecycle phases automate** the entire build process, from validation to deployment.
- **Maven Wrapper** enables running Maven commands even if Maven is not installed system-wide.
- The **build output is a runnable JAR file** that can be executed with just Java installed.
- **Cleaning (mvn clean) and packaging (mvn package)** are common Maven commands used frequently in development.